Chapter 2 Exercises

Use traditional pre-Release-6 MIPS32 with one explicit `nop` delay slot after each control transfer. `jal` saves $PC + 8$. Show fields, register numbers, addresses, and assumptions.

CH02-Q01 Encoding

1. Encode `sll $t0, $t1, 3` as a hexadecimal 32-bit word.
2. Encode `addiu $s0, $sp, -16`.
3. Give each format, opcode, main fields, and extension rule.

CH02-Q02 Two's complement and overflow

Registers are 32 bits.

1. Interpret `0xfffffff6` as unsigned and signed decimal values.
2. Compute the low 32 bits and signed overflow for `0x7fffffff + 1`; distinguish the actual MIPS `add` and `addu` effects.
3. Give result bits and unsigned overflow for `0xffffffff + 1`.
4. Compare `0xffffffff` and `0x00000001` under signed and unsigned ordering. Explain which instruction fits each interpretation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH02-Q03 Loop invariant

The following code sums absolute values of signed array elements. `$a0` points to aligned words, `$a1=n`, $0 \leq n \leq$ array length, and for n>0 `$a0+4(n-1)` does not wrap. Assume each lw is readable and jumps are encodable; for the n-iteration proof also assume no INT_MIN and no other exception. Both counter and accumulator are explicitly initialized.

```
addiu $t0, $zero, 0
addu  $v0, $zero, $zero
loop:
beq  $t0, $a1, done
nop
sll  $t1, $t0, 2
addu $t1, $t1, $a0
lw   $t2, 0($t1)
slt  $t3, $t2, $zero
beq  $t3, $zero, skip
nop
sub  $t2, $zero, $t2
skip:
addu $v0, $v0, $t2
addiu $t0, $t0, 1
j    loop
nop
done:
```

1. State the invariant at each entry to the loop body.
2. Use initialization, branch conditions, delay slots, and the increment to prove exactly `n` body executions when no exception occurs.
3. What happens for `$t2=0x80000000`?
4. Which instructions trap or wrap if the signed sum exceeds its range?

CH02-Q04 Nested calls and a stack frame

Under an o32-like convention with 8-byte stack alignment and an argument home area, `outer` receives one argument, modifies `$s0`, and calls `inner`. Its frame is 32 bytes. Give a conceptual sequence without implementing `inner`.

1. Which state must `outer` preserve under caller/callee-save rules?
2. Show entry, argument preparation, call continuation, and exit operations.
3. How should the frame handle a second sequential `jal inner`?
4. Which obligation does `inner` violate if it destroys `$s0` without restoring it?

CH02-Q05 Target addresses

Use 32-bit byte addresses and aligned instructions.

1. A branch has `PC+4 = 0x00400010` and `imm16 = 0xfffa`. Compute its target.
2. A J-format instruction has `PC+4 = 0x0abcdef4` and `target26 = 0x0004000`. Compute its target.
3. Can that instruction directly target `0x1abcdef4`?
4. How many bytes does branch immediate `0x0002` denote?

CH02-Q06 Atomic operations

Two cores each increment the counter at `$s0` once.

```
retry:
ll    $t0, 0($s0)
addiu $t0, $t0, 1
sc    $t0, 0($s0)
beq   $t0, $zero, retry
nop
```

1. Why is ordinary load, increment, and store insufficient?
2. What does failed `sc` leave in memory and in its register operand?
3. What happens when an intervening store invalidates the reservation?
4. Does atomicity guarantee starvation freedom? What further policies might be needed?

CH02-Q07 Toolchain

`main.o` calls `dot` in `math.o`. That object calls `axpy` in `libvec.so`.

1. Allocate reference resolution among assembler, static linker, and dynamic loader.
2. Why are relocation records necessary?
3. Distinguish file offset from runtime virtual address.
4. Explain the range issue for a PC-relative call when a library is far away.

CH02-Q08 ISA and implementation

A executes 1.8×10^9 instructions at CPI 1.20 and 3.0 GHz. B adds a complex instruction, reducing count to 1.5×10^9, but CPI rises to 1.55 at the same frequency.

1. Compute both times.
2. Compute B's speedup relative to A.
3. Why does a 16.7% count reduction not prove a win?
4. Find B's break-even frequency under constant CPI.

CH02-Q09 Decoding and control transfers

Use classic MIPS32 with one delay slot after every control transfer.

1. Decode `0x8fa8000c`. Give its format, destination, base, and effective address.
2. Decode `0x1109fffc` at `0x00400020`, including both operands and immediate. Give complete taken and not-taken PC sequences.
3. For `0x0c100140` at `0x00400070`, find target, delay-slot PC, and `$ra`.
4. What wrong target results from treating the branch immediate as a byte displacement?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH02-Q10 Layout, sorting, and cost

```
struct row { int64_t id; int32_t vals[4]; int64_t sum; };
```

Use default System V AMD64 alignment without packing, with at least six allocated rows. The base of `rows` is `0x3000` in `%rdx`. Let `i=5` in `%rdi` and `j=3` in `%esi`.

1. Find structure size, `vals` offset, and the address of `rows[5].vals[3]`.
2. Give a `movslq`, `salq`, and `leaq` sequence and explain why `lea` does not read memory.
3. Trace every inner comparison and array state of the adjacent-swap insertion sort below on `[3,1,2]`.

```
for i = 1 to n-1:
    j = i
    while j > 0 and v[j-1] > v[j]:
        swap(v[j-1], v[j])
        j = j - 1
```

4. Under the common word-size and delay-slot model, indexed clear uses seven steady-state instruction positions per iteration and pointer clear five. Which costs are excluded? Why may compiler strength reduction make their machine code similar?

References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 2. CSO course slides, Chapter 2.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study